

NOTICE

- Deadlines
 - (5/20 9:00 AM) PA3 Grades will be released
 - (6/01 8:00 AM) Midterm Quiz 4
 - (6/12 5:00 PM) PA 5 deadline
 - (6/12 5:00 PM) PA 1–4 late submission deadlines

CS 374: OPERATING SYSTEMS I

PART IV – SYNCHRONIZATION III

Mon/Wed 12:00 – 1:50 PM

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI

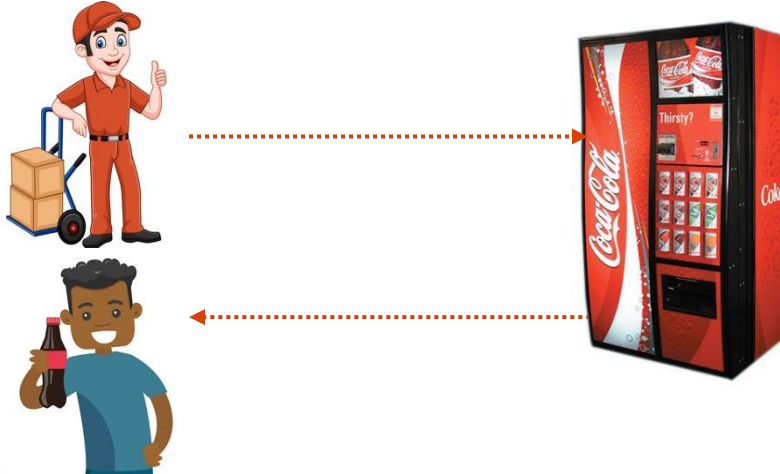
TOPICS FOR TODAY

- Part IV – Synchronization III
 - Recap:
 - Coke machine
 - Semaphore
 - Manage resources
 - Meeting room booking system
 - Semaphore and its limit
 - Provide abstraction & Offer standard interface
 - Monitor
 - Monitor in C (Meeting room booking)
 - Monitor in C (Coke machine)

PRODUCER-CONSUMER PROBLEM

- A coke machine

- Two workers (or threads):
 - Producer: fills the coke machine
 - Producer needs to wait if the machine is full
 - Consumer: takes cokes from the machine
 - Consumer needs to wait if it is empty



COKE MACHINE V0.2

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- **Deadlock:**

- Producer enqueue
- It locks and wait for an empty slot(s)
- Consumer waits “unlock”

```
#define MACHINE_CAPACITY 64
```

```
static struct coke_machine;
```

```
void producer_fn() {  
    while (1) {  
        pthread_mutex_lock(&machine);  
        while (machine == full) {};  
        enqueue(acoke, coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
void consumer_fn() {  
    while (1) {  
        pthread_mutex_lock(&machine);  
        while (machine == empty) {};  
        acoke = dequeue(coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
int main(void) {  
    pthread_t producer, consumer;
```

```
....
```

```
    return 0;
```

```
}
```

```
// code only reaches here if the machine is broken
```

COKE MACHINE V0.2

- **Deadlock**

- **Definition:** a “state” where:

- 2+ threads are each waiting for a resource held by the other
 - none can make progress

- **Deadlock happens**

- **Mutex:** a resource can be used by only one thread at a time
 - **Hold and wait:** a thread hold the resource while waiting for another
 - **No preemption:** a resource cannot be forcibly taken by a thread
 - **Circular wait:** threads form a cycle, each waiting for the next thread’s resource

- **Solution:** break one of the four conditions above

COKE MACHINE V0.2

- **Deadlock**

- **Definition:** a “state” where:
 - 2+ threads are each waiting for a resource held by the other
 - none can make progress

- **Starvation**

- **Definition:** one specific thread keeps getting not scheduled (others proceed fine)

COKE MACHINE V0.3

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- **Solution III:**

- *Break hold and wait*
- Busy-wait “consumes CPUs”
- Not sure when busy-wait threads will run
 - Multiple threads run on CPUs
 - Starvation

```
#define MACHINE_CAPACITY 64
```

```
static struct coke_machine;
```

```
void producer_fn() {  
    while (1) {  
        while (machine == full) {};  
        pthread_mutex_lock(&machine);  
        enqueue(acoke, coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
void consumer_fn() {  
    while (1) {  
        while (machine == empty) {};  
        pthread_mutex_lock(&machine);  
        acoke = dequeue(coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
int main(void) {  
    pthread_t producer, consumer;
```

```
....
```

```
    return 0; // code only reaches here if the machine is broken
```

```
}
```


HOW TO ADDRESS BUSY-WAITING PROBLEMS

- Semaphore

- **Definition:** a synchronization object with a counter and a waiting queue
- **TL; DR:** Mutex + Counter + Sleep/Wake mechanism

- Semaphore operations

- **P():** *decrease* the counter by 1; will *wait* until a semaphore becomes positive
- **V():** *increase* the counter by 1; will *wake up* any thread that waits by P()

P() or sem_wait():

sem.value -= 1

if sem.value < 0:

add thread to sem.queue

sleep() // remove

V() or sem_post():

sem.value += 1

if sem.value >= 0:

pick one thread from sem.queue

wake_up() // move to ready Q

BUSY-WAIT VS. SEMAPHORE

- Busy-wait
 - Thread checks the while condition
 - If the resource is not yet available
 - It checks the while condition again
 - ... (do this forever... which burns CPU time)
- Semaphore
 - Thread is put into sleep by `sem_wait()`
 - If the condition has not been met, *it sleeps and yields CPUs*
 - Another thread calls `sem_post()`
 - If the condition is met, it proceeds... (no resource starvation)

HOW TO ADDRESS BUSY-WAITING PROBLEMS

- Semaphore POSIX API

- `sem_t` semaphore: semaphore object
- `int sem_init(sem_t *sem, int pshared, unsigned int value)`
 - `pshared`: 0 – shared between a process' threads | 1 – shared between processes
 - `value`: counter value
- `int sem_destroy(sem_t *sem)`
- `int sem_wait(sem_t *sem)` : P()
- `int sem_post(sem_t *sem)` : V()
- `int sem_trywait(sem_t *sem)` : Does not wait if no resource is available
- `int sem_getvalue(sem_t *sem, int *val)`

- Note

- Need to link `-lpthread` in compilation (e.g., `gcc -o main main.c -lpthread`)

BINARY VS. COUNTING SEMAPHORE

- **Binary semaphore**

- Semaphore with counter 1 (operates like mutex)
- But threads cannot own the lock
- *Not a mutex*; mutex owned by a thread

```
sem_t lock;  
sem_init(&lock, 0, 1)  
...  
sem_wait(&lock)  
(...critical section...)  
sem_post(&lock)
```

- **Counting semaphore**

- Semaphore with counter N
- Manages N resources at the same time

```
sem_t lock;  
sem_init(&lock, 0, 64)  
...  
sem_wait(&lock)  
(...critical section or not...)  
sem_post(&lock)
```

COKE MACHINE V0.4

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- **Solution IV:**

- Counting semaphore

```
sem_t slots_filled;
sem_t slots_empty;

void producer_fn() {
    while (1) {
        sem_wait(&slots_empty);

        enqueue(acoke, coke_machine);

        sem_post(&slots_filled);
    }
}

void consumer_fn() {
    while (1) {
        sem_wait(&slots_filled);
        acoke = dequeue(coke_machine);

        sem_post(&slots_empty);
    }
}

int main(void) {
    int ret;
    ret = sem_init(&slots_empty, 0, 64);
    ret = sem_init(&slots_filled, 0, 0);

    ....
}
```

It decreases “filled slot” by one

It increases “empty slot” by one, and wakes up any thread (i.e., producer thread) by sending a signal to that thread

COKE MACHINE V0.4 – CONT'D

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- **Solution IV:**

- Counting semaphore
- Binary semaphore

```
sem_t mutex;  
sem_t slots_filled;  
sem_t slots_empty;
```

```
void producer_fn() {  
    while (1) {  
        sem_wait(&slots_empty);  
        sem_wait(&mutex);  
        enqueue(acoke, coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_filled);  
    }  
}
```

The semaphore only allows one thread to enqueue (or dequeue)

```
void consumer_fn() {  
    while (1) {  
        sem_wait(&slots_filled);  
        sem_wait(&mutex);  
        acoke = dequeue(coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_empty);  
    }  
}
```

```
int main(void) {  
    int ret;  
    ret = sem_init(&slots_empty, 0, 64);  
    ret = sem_init(&slots_filled, 0, 0);  
    ret = sem_init(&mutex, 0, 1);  
    ....  
}
```

DEADLOCK POSSIBLE WITH SEMAPHORE

- Order matters
 - Switch `sem_wait()`s in Producer
 - Producer locks “mutex”
 - Producer waits for “empty slot”
 - Consumer can’t dequeue b/c of “mutex”
 - **Deadlock!**
 - Switch `sem_post()`s in Producer
 - Producer decreases “slots_filled”
 - Consumer wakes up
 - Consumer can’t dequeue
 - Producer unlocks the mutex
 - Runs... (no deadlock)

Care Must Be Taken!

```
sem_t mutex;  
sem_t slots_filled;  
sem_t slots_empty;
```

```
void producer_fn() {  
    while (1) {  
        sem_wait(&mutex);  
        sem_wait(&slots_empty);  
        enqueue(acoke, coke_machine);  
        sem_post(&slots_filled);  
        sem_post(&mutex);  
    }  
}
```

```
void consumer_fn() {  
    while (1) {  
        sem_wait(&slots_filled);  
        sem_wait(&mutex);  
        acoke = dequeue(coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_empty);  
    }  
}
```

```
int main(void) {  
    int ret;  
    ret = sem_init(&mutex, 0, 1);  
    ret = sem_init(&slots_empty, 0, MACHINE_CAPACITY);  
    ret = sem_init(&slots_filled, 0, 0);
```

```
..... // omit the pthread_create / _join  
}
```

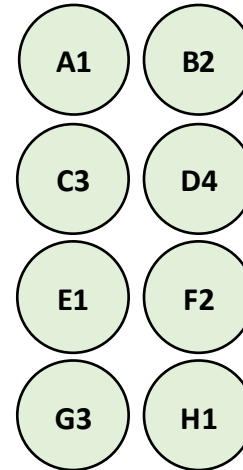
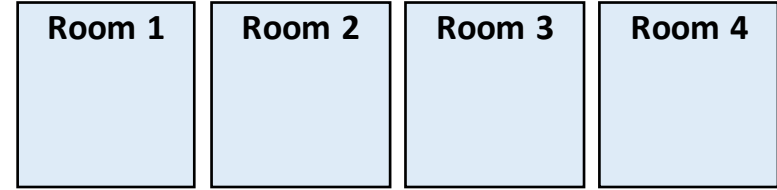
TOPICS FOR TODAY

- Part IV – Synchronization III
 - Recap:
 - Coke machine
 - Semaphore
 - Manage resources
 - Meeting room booking system
 - Semaphore and its limit
 - Provide abstraction & Offer standard interface
 - Monitor
 - Monitor in C (Meeting room booking)
 - Monitor in C (Coke machine)

MEETING ROOM BOOKING SYSTEM

- **Scenario**

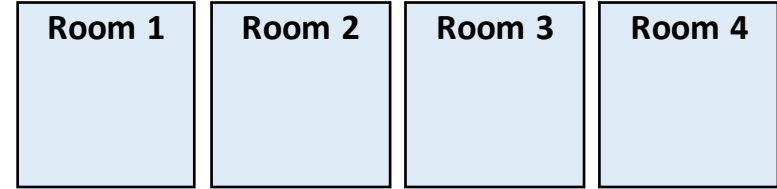
- 4 meeting rooms available
- Only one person can be in a room
- No room left: anyone should wait



MEETING ROOM BOOKING SYSTEM: SEMAPHORE

- **Solution**

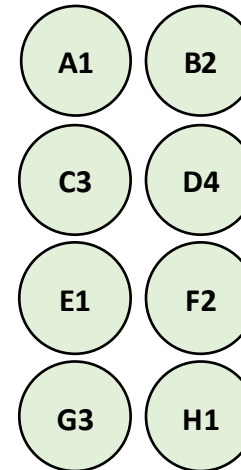
- Semaphore
- 4 conditional variables := 4 rooms



Mutex: no one can enter if someone is in the room

- **Illustrative example:**

- 4 employees (E-H) should wait



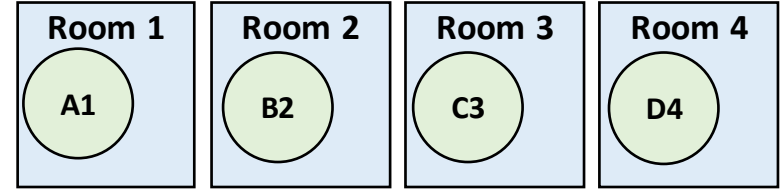
Semaphore

> Conditional var: **0**
(No rooms are available)

MEETING ROOM BOOKING SYSTEM: SEMAPHORE

- **Solution**

- Semaphore
- 4 conditional variables := 4 rooms



Mutex: no one can enter if someone is in the room

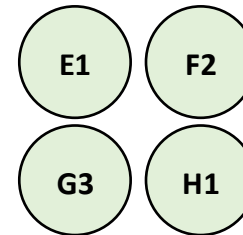
- **Illustrative example:**

- A moves out from Room 1
- Semaphore increases from 0 to 1
- It signals to all waiting process (E-H)
- OS picks one (H) to run
- H enters Room 1
- Semaphore decreases to 0

Semaphore

> Conditional var: **0**

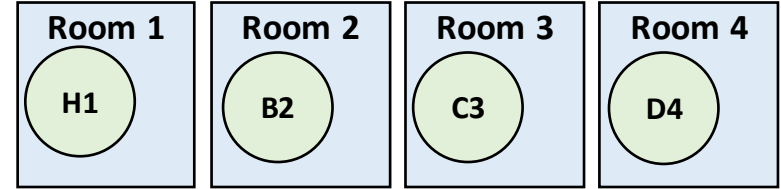
Signal: once Room A is empty, OS lets others (E, F, G, H) its empty



MEETING ROOM BOOKING SYSTEM: SEMAPHORE

- **Solution**

- Semaphore
- 4 conditional variables := 4 rooms



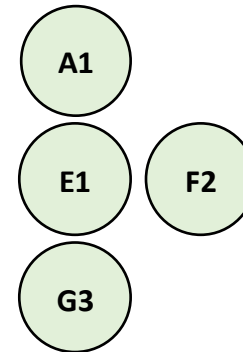
Mutex: no one can enter if someone is in the room

- **Illustrative example:**

- B moves out from Room 2
- Semaphore increases from 0 to 1
- It signals to all waiting process (E-H)
- OS picks one (H) to run
- H enters Room 1
- Semaphore decreases to 0

Semaphore

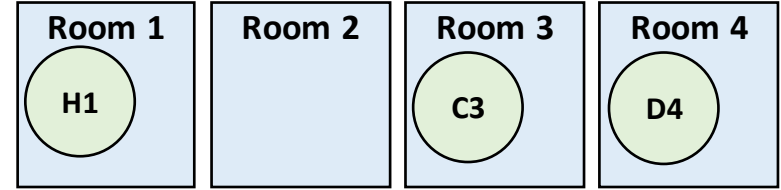
> Conditional var: **0**



MEETING ROOM BOOKING SYSTEM: SEMAPHORE

- **Solution**

- **Semaphore**
- 4 conditional variables := 4 rooms



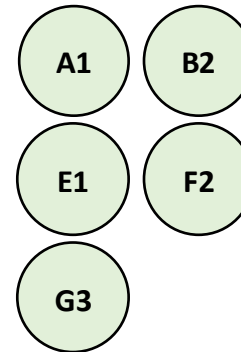
Mutex: no one can enter if someone is in the room

- **Illustrative example:**

- B moves out from Room 2
- Semaphore increases from 0 to 1
- It signals to all waiting process (E-H)
- OS picks one (E) to run

Semaphore

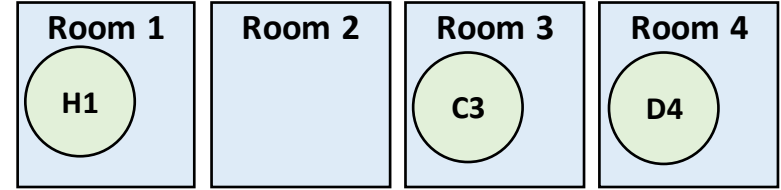
> Conditional var: **1**



MEETING ROOM BOOKING SYSTEM: SEMAPHORE

- **Solution**

- Semaphore
- 4 conditional variables := 4 rooms



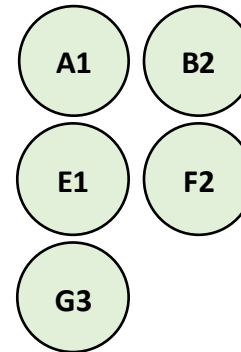
Mutex: no one can enter if someone is in the room

- **Illustrative example:**

- B moves out from Room 2
- Semaphore increases from 0 to 1
- It signals to all waiting process (E-H)
- OS picks one (E) to run
- E enters Room 2!
- Semaphore will decrease to 0!

Semaphore

> Conditional var: **1**



SEMAPHORE IS GOOD, BUT DIFFICULT TO EXPRESS SCHEDULING CONDITIONS

- **Problems:**

- Counter-based
 - Cannot express conditions (e.g., Room 2 is available now)
- Lock and scheduling are coupled
 - cannot control who should wake up
 - You only want to wake “F” as they need Room 2
 - But semaphore let’s the OS decide who runs next
- One ordering mistake
 - Immediate deadlock
 - If the scheduling logic is complex, it is notorious to debug

TOPICS FOR TODAY

- Part IV – Synchronization III
 - Recap:
 - Coke machine
 - Semaphore
 - Manage resources
 - Meeting room booking system
 - Semaphore and its limit
 - Provide abstraction & Offer standard interface
 - Monitor
 - Monitor in C (Meeting room booking)
 - Monitor in C (Coke machine)

DECOMPOSE LOCK AND SCHEDULING

- Monitor:
 - **Definition:** a synchronization *object*
 - *Lock (mutex)*
 - *Conditional variable*
 - Monitoring mechanism
 - “We” implement the monitor: user-level object

MONITOR OPERATIONS

- Monitor:
 - **Definition:** a synchronization *object*
 - *Lock (mutex)*
 - *Conditional variable*
 - Monitoring mechanism
 - **Three operations**
 - `wait(&lock)` : release lock and sleep; obtain lock once wake-up
 - `signal()` : wake up one waiting thread
 - `broadcast()` : wake up *all* waiting jobs

MONITOR OPERATIONS

- Monitor:

- **Definition:** a synchronization *object*

- *Lock (mutex)*
 - *Conditional variable*
 - Monitoring mechanism

- **Three operations**

- `wait(&lock)` : release lock and sleep; obtain lock once wake-up
 - `signal()` : wake up one waiting thread
 - `broadcast()` : wake up *all* waiting jobs

Raise a problem if separated:

`unlock()`

// if a signal comes at this point

// the signal will be blocked

`lock()`

MONITOR SKELETON

- Monitor:

- **Definition:** a synchronization *object*

- Lock (*mutex*)
 - Conditional variable
 - Monitoring mechanism

- **Three operations**

- wait(&lock) : release lock and sleep;
obtain lock once wake-up
 - signal() : wake up one waiting thread
 - broadcast() : wake up *all* waiting jobs

```
static lock monitor_lock;           // lock
static struct queue wait_queue;     // cond variable
static struct room_t meeting_rooms[4];
```

```
-----
acquire(&monitor_lock);

while (meeting_rooms[room_num] != empty) {
    wait(&wait_queue, &monitor_lock);
}

do_work();

signal(&wait_queue);

release(&monitor_lock);
```

MONITOR SKELETON

- Monitor:

- **Definition:** a synchronization *object*

- *Lock (mutex)*
 - *Conditional variable*
 - Monitoring mechanism

- **Three operations**

- wait(&lock) : release lock and sleep;
obtain lock once wake-up
 - signal() : wake up one waiting thread
 - broadcast() : wake up *all* waiting jobs

- **Note**

- broadcast() over signal(): we do not know who is waiting for which room

```
void reserve_a_room(  
    int room_num, struct user_t* employee) {  
  
    acquire(&monitor_lock);  
  
    while (meeting_rooms[room_num] != empty) {  
        wait(&wait_queue, &monitor_lock);  
    }  
  
    rooms[room_num].occupied = 1;  
  
    release(&monitor_lock);  
}  
-----  
struct user_t* release_a_room(int room_num) {  
    acquire(&monitor_lock);  
  
    rooms[room_num].occupied = 0;  
  
    broadcast(&wait_queue);  
  
    release(&monitor_lock);  
}
```

MONITOR SKELETON

- Monitor:
 - **Definition:** a synchronization *object*
 - *Lock (mutex)*
 - *Conditional variable*
 - Monitoring mechanism
 - **Three operations**
 - wait(&lock) : release lock and sleep;
obtain lock once wake-up
 - signal() : wake up one waiting thread
 - broadcast() : wake up *all* waiting jobs
 - **Note**
 - broadcast() over signal(): we do not know who is waiting for which room

```
static lock monitor_lock;
static struct queue wait_queue;
static struct room_t meeting_rooms[4];

-----

void reserve_a_room(
    int room_num, struct user_t* employee) {

    acquire(&monitor_lock);
    while (meeting_rooms[room_num] != empty) {
        wait(&wait_queue, &monitor_lock);
    }
    room_assign(room_num, employee);
    release(&monitor_lock);
}

struct user_t* release_a_room(int room_num) {
    acquire(&monitor_lock);
    employee = room_empty(room_num);
    broadcast(&wait_queue);
    release(&monitor_lock);
    return employee;
}
```

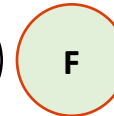
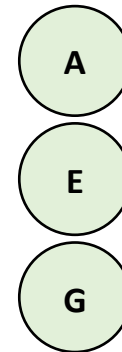
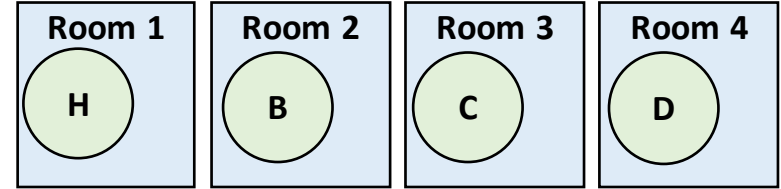
MONITOR IN OPERATION

```
static lock monitor_lock;           // lock
static struct queue wait_queue;     // conditional variable
static struct room_t meeting_rooms[4];

void reserve_a_room(int room_num, struct user_t* employee) {
    acquire(&monitor_lock);
    while (meeting_rooms[room_num] != empty) {
        wait(&wait_queue, &monitor_lock); // wait + unlock + sleep
    }
    room_assign(room_num, employee);
    release(&monitor_lock);
}

struct user_t* release_a_room(int room_num) {
    acquire(&monitor_lock);
    employee = room_empty(room_num);
    broadcast(&wait_queue); // wake up one of them
    release(&monitor_lock);
    return employee;
}
```

←..... Runs



Queue

Monitor

- > A lock
- > A conditional var (queue)
- > Required functions
 - room_reserve()
 - room_release()

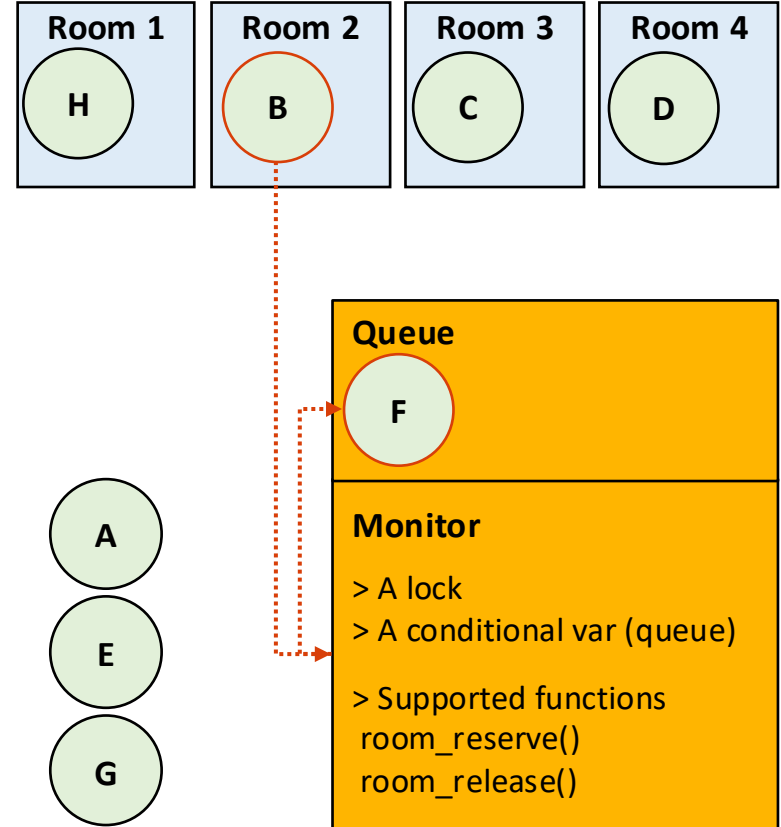
MONITOR IN OPERATION

```
static lock monitor_lock;           // lock
static struct queue wait_queue;     // conditional variable
static struct room_t meeting_rooms[4];

void reserve_a_room(int room_num, struct user_t* employee) {
    acquire(&monitor_lock);
    while (meeting_rooms[room_num] != empty) {
        wait(&wait_queue, &monitor_lock); // wait + unlock + sleep
    }
    room_assign(room_num, employee);
    release(&monitor_lock);
}

struct user_t* release_a_room(int room_num) {
    acquire(&monitor_lock);
    employee = room_empty(room_num);
    broadcast(&wait_queue); // wake up one of them
    release(&monitor_lock);
    return employee;
}
```

←..... Runs



MONITOR IN C

- Previously (abstraction)

- lock monitor_lock
- queue wait_queue
- acquire(&lock)
- release(&lock)
- wait(&queue, &lock)
- signal(&queue)
- broadcast(&queue)

- C Pthread API

- pthread_mutex_t monitor_lock
- pthread_cond_t wait_queue
- pthread_mutex_lock(&lock)
- pthread_mutex_unlock(&lock)
- pthread_cond_wait(&cond, &lock)
- pthread_cond_signal(&cond)
- pthread_cond_broadcast(&cond)

COKE MACHINE V0.5

• Monitor version

- Mutex lock
- Conditional variables
 - Producer
 - Consumer
- signal() over broadcast()

```
#include <pthread.h>
#define NUM_SLOTS 64
```

```
typedef struct { int id; } coke_t;
```

```
static pthread_mutex_t machine_lock = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t producer_cond = PTHREAD_COND_INITIALIZER;
static pthread_cond_t consumer_cond = PTHREAD_COND_INITIALIZER;
```

```
static coke_t buffer[NUM_SLOTS];
static int head = 0, tail = 0, count = 0;
```

```
void produce(int producer_id) {
    pthread_mutex_lock(&machine_lock);

    while (count == NUM_SLOTS) {
        pthread_cond_wait(&producer_cond, &machine_lock);
    }

    buffer[tail] = (coke_t){ .id = producer_id };
    tail = (tail + 1) % NUM_SLOTS;
    count++;

    pthread_cond_signal(&consumer_cond);
    pthread_mutex_unlock(&machine_lock);
}
```

```
coke_t consume() {
    pthread_mutex_lock(&machine_lock);

    while (count == 0) {
        pthread_cond_wait(&consumer_cond, &machine_lock);
    }

    coke_t coke = buffer[head];
    head = (head + 1) % NUM_SLOTS;
    count--;

    pthread_cond_signal(&producer_cond);
    pthread_mutex_unlock(&machine_lock);
    return coke;
}
```

COKE MACHINE V0.5 - MONITOR

- **Misuse case I**

- A shared conditional variable (below)
- signal() can wake up the Producer
- Producer will sleep (count == #SLOTS)
- Consumer waits
- **No progress!**

```
void produce(int producer_id) {
    pthread_mutex_lock(&machine_lock);

    while (count == NUM_SLOTS) {
        pthread_cond_wait(&producer_cond, &machine_lock);
    }

    buffer[tail] = (coke_t){ .id = producer_id };
    tail = (tail + 1) % NUM_SLOTS;
    count++;

    pthread_cond_signal(&consumer_cond);
    pthread_mutex_unlock(&machine_lock);
}

-----
static pthread_cond_t shared_cond;

void produce(...) {
    pthread_mutex_lock(&lock);
    while (count == NUM_SLOTS)
        pthread_cond_wait(&shared_cond, &lock);

    pthread_cond_signal(&shared_cond);
    pthread_mutex_unlock(&lock);
}
```

COKE MACHINE V0.5 - MONITOR

- **Misuse case II**

- if instead of **while**
- A thread wakes up
- if condition has checked before sleep
- Proceed without any check
- **Race condition!**

- **Misuse case III**

- Update the room status without lock
- **Race condition!**

```
void reserve_a_room(  
    int room_num, struct user_t* employee) {  
  
    acquire(&monitor_lock);  
    if (meeting_rooms[room_num] != empty) {  
        wait(&wait_queue, &monitor_lock);  
    }  
    room_assign(room_num, employee);  
    release(&monitor_lock);  
}
```

```
-----  
  
struct user_t* release_a_room(int room_num) {  
    employee = room_empty(room_num);  
    acquire(&monitor_lock);  
    broadcast(&wait_queue);  
    release(&monitor_lock);  
    return employee;  
}
```

TOPICS FOR TODAY

- Part IV – Synchronization III
 - Recap:
 - Coke machine
 - Semaphore
 - Manage resources
 - Meeting room booking system
 - Semaphore and its limit
 - Provide abstraction & Offer standard interface
 - Monitor
 - Monitor in C (Meeting room booking)
 - Monitor in C (Coke machine)

Thank You!

Mon/Wed 12:00 – 1:50 PM

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI